

Cognizant - DN 5.0

- ① Design Pattern and Principle,
- ② Data Structure and Algorithm,
- ③ PL/SQL Programming
- ④ Test driven development and logging framework

• What is a basically design?

For real time project development
↳ will follow some design principle, design pattern.

What is design Principle?

↓
What is basically design?

- SDLC Phases:
- ① Requirement document, ^{Business Requirement Development} BRD → Analysis phase
 - ② What technology - core java, spring boot, rest api JPA / hibernate, MySQL/oracle + hibernate, Design phase - Low level Design, HLD, system design
 - ③ Development - writing code
 - ④ Testing
 - ⑤ Deploy - Server

Design is basically going out with all SDLC phase.

___/___/___

Design Principle :

During the development of applications the principle which we have to take care of is called design principle.

for eg: As per development

design principle : SOLID principle

Because individual development $\rightarrow$ 1-2 people

But project development $\rightarrow$ 100-200 people

If we will not follow any design principle the project success rate is low.

* Clean Code fundamentals:

- Short method names
- One method = one job
- No unused variable
- Proper indentation
- Easy to test

```
public class Calculator {
```

```
    public int add (int a, int b) {
```

```
        return a + b;
```

```
    }
```

```
}
```

SOLID

S → Single Responsibility Principle

O → Open closed Principle

L → Liskov substitute principle

I → Interface segregation

D → Dependency Inversion

- [In open closed Principle]

```
interface Shape { double area(); }
```

Class Circle implements Shape {

private double r;

Circle (double r) { this.r = r; }

public double area() { return Math.PI * r * r; }

}

}

closed for modification but open for extension

- Single Responsibility Principle

```
class InvoicePrinter {
```

public void print (String invoice) {

System.out.println (invoice);

}

}

- Class does one task

- Method is focused

- Easy to extend

- Simple testing

- Substitution Method

```
class Bird {
```

```
    public void fly() {
```

```
        System.out.println("Flying");
```

```
    }
```

- `class sparrow extends Birds {`

- Subclass replaced parent

- It comes under inheritance

↓
child class extends
super class.

- Interface Segregation

→ Developing the code for application, we need to write multiple interfaces.

- small interfaces
- Only need methods
- No empty implementation
- Flexible design

- Dependency inversion
 - Depends on interface
 - Inject service via constructor
 - No hard-coded dependency
 - Easy to swap implementation

• What is classically design pattern?

↗ Why pattern?
 ↘ What is pattern?

Design Patterns (Structural, Creational, Behavioral)
 Singleton JVM

What is problem?

↳ multiple objects are created

What is solution?

↳ we need to restrict object -- only 1 object

So there is solution for repeating problem which is come to dr. design patterns.

- ① Creational -- instantiation (object creation)
- ② Structural -- Object composition
- ③ Behavioral -- Object interaction